An Efficient Multi-signature Wallet in Blockchain Using Bloom Filter*

Jongbeen Han
Seoul National University
Seoul, South Korea
stompesi@gmail.com

Mansub Song
Seoul National University
Seoul, South Korea
ssongman1029@gmail.com

Hyeonsang Eom
Seoul National University
Seoul, South Korea
hseom@snu.ac.kr

Yongseok Son
Chung-Ang University
Seoul, South Korea
sysganda@cau.ac.kr

ABSTRACT

A blockchain wallet is a financial application which provides an interface for managing a digital asset. In the blockchain wallet, a digital signature (single or multiple signatures) is used to sign a transaction. However, the wallet is often vulnerable to security attacks when a single-signature scheme is used, and there are performance issues when a multi-signature scheme is employed. To address these issues, we propose a new efficient multi-signature wallet that improves performance, storage efficiency, and privacy without modifying the blockchain protocol. The steps in the development of the wallet are as follows, we first design a blockchain wallet that supports multi-signature as a single-signature by using a threshold elliptic curve digital signature algorithm (T-ECDSA) for high validation performance. Second, for achieving high storage efficiency and privacy, we use a bloom-filter for transactions for ensuring a small transaction size and identifying the participant of a transactions without exposing participant information. Experimental results indicate that the proposed multi-signature wallet shows higher verification performance in a blockchain, with a small transaction size compared with existing wallets.

KEYWORDS

Multi-signature, Bloom-filter, Blockchain, T-ECDSA, Blockchain wallet

ACM Reference Format:
Jongbeen Han, Mansub Song, Hyeonsang Eom, and Yongseok Son. 2021. An Efficient Multi-signature Wallet in Blockchain Using Bloom Filter. In *The 36th ACM/SIGAPP Symposium on Applied Computing (SAC '21), March 22–26, 2021, Virtual Event, Republic of Korea.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3412841.3441910

*Produces the permission block, and copyright information

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SAC '21, March 22–26, 2021, Virtual Event, Republic of Korea
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8104-8/21/03…$15.00
https://doi.org/10.1145/3412841.3441910

1 INTRODUCTION

Blockchains have become popular since they distribute data while maintaining the integrity of data, support byzantine fault tolerance (BFT), and provide inter-individual transfers without requiring any intermediary entities [16, 22, 26]. One of the blockchain features is a digital signature that provides non-repudiation. More specifically, blockchain wallets support digital signatures by using keys (i.e., private and public keys). The keys are used to sign a transaction and verify the identity of the transaction signer on the blockchain.

There are two types of digital signature schemes: single-signature and multi-signature schemes. A single-signature scheme is a digital signature scheme that allows only one user to agree to a transaction by own sign. Therefore, this scheme can be vulnerable to security attacks due to a single-point attacks such as a single point attack. By contrast, a multi-signature scheme is a digital signature scheme that allows a group of users to agree to a transaction by their sign [13]. Accordingly, the multi-signature usually generates multiple signatures that are to be validated and includes all participants' public keys in a transaction for validating the transaction. While the multi-signature scheme provides higher security than a single-signature scheme by distributing the authority for signing a transaction. It shows a lower performance for validating signed transactions and larger transaction sizes since multiple signatures are to be made and validated.

Previous studies [3, 4] have investigated multi-signature on blockchains to increase the blockchain wallet's security. BitGo [3] proposes pay to script hash (P2SH), which is a new type of Bitcoin address introduced as part of Bitcoin improvement proposal (BIP) 16 for supporting multi-signature on Bitcoin. MultiSigWallet [4] is based on a smart contract and provides high security by allowing multiple parties to agree on transactions. The study reported herein use these approaches [3, 4] in terms of investigating a blockchain wallet. We focus on providing high performance of multi-signature and storage efficiency on blockchains by reducing the number of transactions to be validated and the transaction size.

In this paper, we propose a new efficient multi-signature wallet with improved performance, storage efficiency, and privacy compared with existing wallets; the proposed wallet does not require any modification of the blockchain protocol. Our goal is to achieve (1) a security level superior to that of a single-signature scheme and (2) performance and storage efficiency higher than those of existing multi-signature schemes. To do this, for improving the

performance (i.e., transaction validation performance), we first design a blockchain wallet in which a set of multiple signatures could be treated as a single signature by using a threshold elliptic curve digital signature algorithm (T-ECDSA) [17]. For example, unlike a multi-signature wallet, our proposed wallet generates a single signature by reconstructing multiple participants' signatures by using T-ECDSA and it allows a blockchain to validate the single signature. Consequently, it can provide high validation performance on blockchains owing to the advantage of using a single signature.

Second, to enhance privacy and storage efficiency, we employ a Bloom-filter [15] on a transaction to identify each transaction's participant with a small transaction size. For example, we store the Bloom-filter in a signed transaction instead of public keys or addresses (the size of public keys and addresses is larger than that of Bloom-filter). This can also prevent exposure to personal information since the Bloom-filter does not store actual data. Thus, we can provide space-efficiency and a high privacy level by using the Bloom-filter. Furthermore, our proposed multi-signature wallet can be used in any blockchains (e.g., Bitcoin, Ethereum) without modifying the blockchain protocol or using a smart contract.

We implement and apply the proposed multi-signature wallet by using the Multi-party ECDSA [14], which is open-source project. Experimental results demonstrate that the proposed multi-signature wallet improves the performance, storage efficiency, and privacy compared with existing multi-signature wallets. The contributions of our work are as follows:

- We analyze the security and performance of existing blockcha in wallets.
- We propose a new multi-signature wallet that shows better performance, storage efficiency, and privacy than existing blockchain wallets; the proposed wallet involves T-ECDSA and a Bloom-filter and does not require any modification of the blockchain protocol.
- We experimentally demonstrate that the proposed multi-signature wallet shows better transaction validation performance, storage efficiency and cost efficiency compared with existing multi-signature wallets.

The rest of this paper is organized as follows: Section 2 describes the background and motivation. Section 3 presents the design of the proposed scheme and explains its implementation. Section 4 shows the experimental results. Section 5 discusses related work, and Section 6 concludes this paper.

2 BACKGROUND AND MOTIVATION

2.1 Blockchain wallet

A blockchain wallet is a financial application the at provides an interface for managing digital assets (e.g., Bitcoin, Ethereum, etc.). It manages a user's private and public keys for signing a transaction involving the transfer of digital assets. A user can prove that he/she owns the digital assets on the blockchain by signing a transaction with the private key in the wallet. Most blockchain wallets manage digital assets using a single-signature scheme such as the elliptic curve digital signature algorithm (ECDSA) [21]. This algorithm is a variant of the digital signature algorithm (DSA) by using elliptic curve cryptography (ECC). ECC is a public-key cryptography approach that is based on the algebraic structure of elliptic curves over finite fields [7]. The ECDSA is more secure than the Rivest–Shamir–Adleman (RSA) [20] algorithm against current cracking methods such as brute-force attacks owing to its complexity . Consequently, a blockchain wallet based on the ECDSA can sign a transaction with a higher security level and relatively small key size. However, the blockchain wallet has security issues since the private key is susceptible to attack from a single attack point because of the nature of the ECDSA. Therefore, a more secure mechanism is required for blockchain wallets.

2.2 Multi-signature wallet

To compensate for the disadvantage of the single-signature wallet, multi-signature wallet allows a group of users to agree to a transaction by their signs and divides up responsibility for transmission of digital assets among multiple users [13]. Participants signing a transaction can be identified from the participant's public key in the multi-signature wallet. This multi-signature wallet can provide higher security for blockchain transactions, since it distributes the authority of the wallet to several participants. For example, in the case of a single-signature wallet, if only a private key of the wallet is obtained by a malicious attacker, the digital assets can be stolen. Meanwhile, a multi-signature wallet provides higher security than a single-signature wallet because a malicious attacker has to acquire a certain number of private keys; this number of keys is configured when the wallet is generated.

However, the multi-signature wallet has some issues that increase the transaction validation and the transaction size in blockch ains. For example, in a Bitcoin blockchain, a multi-signature wallet signs a transaction while generating multiple signatures, and the signed transaction is verified by the full nodes on the blockchain network. Therefore, it increases the verification time compared with that of a single-signature scheme because multiple signatures should be verified. Moreover, when the multi-signature wallet is used on a blockchain, the blockchain protocol should be modified [1]. Meanwhile, in the case of Ethereum, a smart contract should be used for utilizing the multi-signature wallet [4]. In summary, we devise the proposed multi-signature wallet such that neither does it require the protocol to be modified in the Bitcoin nor does it necessitate the use of a smart contract in the Ethereum.

2.3 Threshold signature scheme

A threshold signature scheme (e.g., T-ECDSA and Shamir's secret sharing scheme) enables n participants to share the power to generate a digital signature with a single public key. A threshold t is specified such that any subset of $t + 1$ participants can jointly sign, but any smaller subset cannot [17, 25]. For example, the T-ECDSA generates a single signature from all the signatures of $t + 1$ participants while Shamir's secret sharing scheme generates one signature by using a private key reconstructed with $t + 1$ shares of participants centrally. Both threshold signature schemes produce a signature that is validated with a common public key and they are compatible with an existing ECDSA that does not require the modification of the blockchain protocol.

Generally, when signing a transaction, the ECDSA generates a signature by using the generation point of an elliptic curve G, a random number k, a hash value of message (e.g., transaction) z,

and a private key pk [21]. As shown in the equation 2, first, the ECDSA generates a point P_{xy} by multiplying the generation point of elliptic curve G and the random number k. The r value is used to generate the s value, and both r and s values are used to validate the transaction. The r value is calculated using the x coordinate of point P (P_x) by modulo n. The parameter n is the integer order of G and is a large prime number. The s is a value generated by a hash value of message (e.g., transaction) z, random k, private key pk and r by a singe user. In the case of the T-ECDSA, k and pk are decided by multiple users rather than a single user, without exposing the user information. For example, the k and pk values can be calculated via shared keys (i.e., k_1/pk_1, k_2/pk_2, ..., k_n/pk_t) between users (i.e., $user_1$, $user_2$, ..., $user_n$). Therefore, this algorithm provides higher security than the ECDSA since it distributes the signing authority.

$$\begin{aligned} P_{xy} &= k \cdot G, r = P_x \ mod \ n \\ s &= k^{-1}(z + rpk) \ mod \ n \end{aligned} \tag{1}$$

Furthermore, the security of the T-ECDSA is identical to that of a multi-signature scheme despite the use of a single signature since it generates several shared keys. Additionally, the verification algorithm of the T-ECDSA is identical to that of a single signature scheme the T-ECDSA generates a single signature. Therefore, we use the T-ECDSA on a blockchain to provide better security and verification performance compared with single-signature and multi-signature schemes, respectively.

2.4 Bloom-filter

A Bloom-filter is a space-efficient probabilistic data structure. It is used to validate whether an element is a member of a set [15]. One of the Bloom-filter features is a false-positive resulting from hash collision. Therefore, a query of the Bloom-filter returns the information that an element is probably in a set or that it is definitely not in a set. A Bloom-filter consists of an array structure of size m bits and uses k different hash functions. A Bloom-filter size can be decided with an array of m-bits and the k hash functions by using an equation 2.

$$\begin{aligned} m &= ceil((N * log(P))/log(1/pow(2, log(2)))) \\ k &= round((m/N) * log(2)) \end{aligned} \tag{2}$$

N is the maximum number of items added to the filter, and P is the probability of false positives. A Bloom-filter involves two processes: adding an element to the filter and validating whether the element is in a set or not. To add an element, a Bloom-filter uses the k hash functions to obtain the k array positions. The Bloom-filter then sets the bit value at all these positions to one on an array of size m bits. To validate whether an element is in a set, the Bloom-filter uses each of the k hash functions to obtain the k array positions. If the bit at any of these position is zero, the element is definitely not in the set; otherwise, it is probably in the set.

We use a Bloom-filter to identify participants in a signed transaction. By storing the Bloom-filter instead of a participant's information (i.e., public key or address) on the blockchain, we can provide higher confidence and reduce the transaction size[1].

[1]Generally, the size of a public key is 64 bytes and the size of an address is 20 bytes. Meanwhile, the size of the Bloom-filter is up to 36 bytes

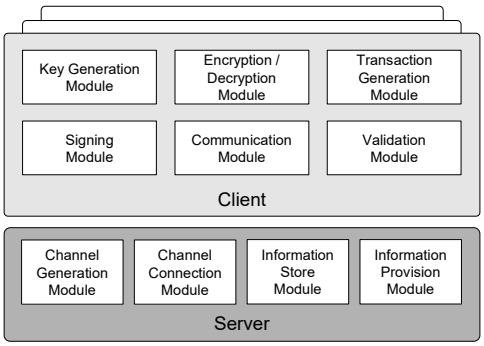


Figure 1: Overall architecture of proposed scheme

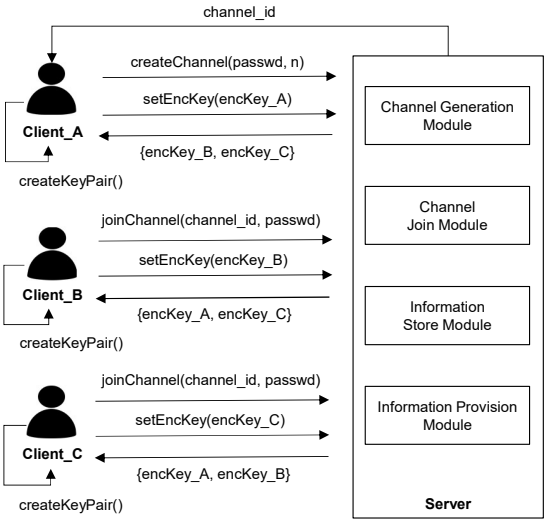


Figure 2: A pre-processing overview

3 DESIGN AND IMPLEMENTATION

To achieve higher performance, storage efficiency, and privacy, we propose a new efficient multi-signature wallet. We aim to increase security compared with a single-signature wallet and improve performance and storage efficiency compared with a multi-signature wallet. To do this, to improve the performance (i.e., transaction validation), we devise a blockchain wallet by exploiting a threshold elliptic curve digital signature algorithm (T-ECDSA) [17]. With T-ECDSA, our proposed wallet can generate a single signature by reconstructing the participant's multiple signatures and validates the generated single signature on blockchains. It provides better validation performance since a single signature validation performance is better than that of multiple signatures.

Second, we exploit a Bloom-filter [15] on a transaction to enhance privacy and the storage efficiency. With the Bloom-filter, we could identify each participant of the single signature generated by the T-ECDSA. This scheme can increase the storage efficiency by storing the Bloom-filter instead of public keys in a signed transaction or addresses (the size of public keys and address are larger than that of the Bloom-filter). This scheme can also prevent exposure of personal information since the Bloom-filter does not store actual data. Finally, our proposed multi-signature wallet can be used in

any blockchain (e.g., Bitcoin, Ethereum) without modifying the blockchain protocol or using a smart contract.

3.1 Overview

Figure 1 shows the overall architecture of the proposed multi-signature wallet which comprises a server and clients (i.e., blockchain wallets). The server supports the exchange of each client's information such as public key, encryption key, and local reconstructing the participants' multiple signatures and validate signature. To do this, we devise four modules such as *channel generation module*, *channel connection module*, *information store module*, and *information provision module* in the server. The *channel generation module* creates a channel for communication between clients. The *channel connection module* allows clients to connect to the created channel to exchange information. For example, the clients can generate a common public key of a multi-signature wallet via the channel[2]. The *information store module* stores the information generated by each client on the server and the *information provision module* transfers the participants' stored information to the clients. Note that all information stored in the server is encrypted, protecting the information against a malicious server administrator.

A client supports to manage private/public keys and signs a transaction. To facilitate this, we devise six modules in the client as shown in Figure 1. The *key generation module* generates private and public keys. It also creates a common public key of a multi-signature wallet by recombining all participant's public keys. The *encryption and decryption module* performs encryption and decryption to prevent the disclosure of the participants' information. The *transaction generation module* generates a blockchain transaction (e.g., Bitcoin and Ethereum). Also, it creates and stores a bloom-filter that represents participants signing a transaction.

The *signing module* signs a transaction by generating a local signature per participant for the transaction and reconstructing the local signatures. This module generates a single signature from multiple local signatures. The *communication module* connects the server to the store or extracts information on clients. The *validation module* verifies whether the client information is forged. If the information is forged, the module stops the process of generating a multi-signature wallet or signing a transaction. Furthermore, this module identifies participants of a transaction by using the Bloom-filter.

3.2 Preparing to exchange information.

Before generating a multi-signature wallet and signing a transaction, clients perform a preprocessing step for secure information exchange. In this step, the clients connect to a server and exchange their own encryption keys with each other. Figure 2 shows the preprocessing step. First, client A requests to create and join a channel with a channel's password (`passwd`) and the number of wallet participants (`n`) to the server via `createChannel()`. The *channel generation module* in the server generates a channel ID (`channel_id`) with the received password and returns the channel ID to the client[3]. After then, the rest of the clients (client B and C) can join the channel with the channel ID and password via `joinChannel()`.

[2]The common public key becomes the representative address of a multi-signature wallet. The address is used to receive and send digital assets on the blockchain network [12].

[3]The channel ID and password will be shared between clients offline

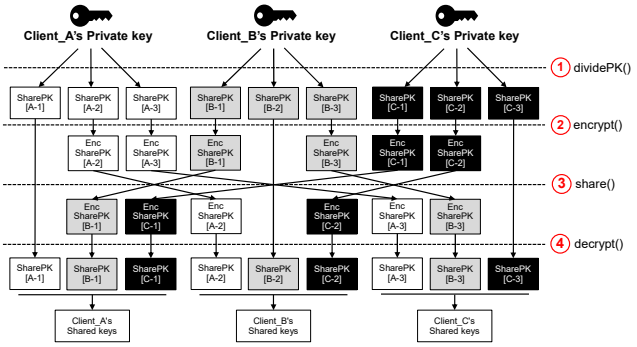


Figure 3: A key generation process for 2 of 3 multi-signature wallet

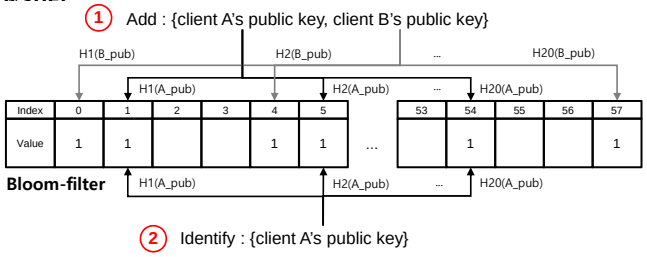


Figure 4: Using bloom-filter in a multi-signature wallet (H: hash function, pub: public key)

After all clients join the channel, each client generates an encryption key (`encKey`) / decryption key (`decKey`) pair via `createKeyPair()` and, uploads the encryption key to the server via `setEncKey()`. The *information store module* stores and manages the encryption key. And then, each client requests the other client's encryption keys to the *information provision module*, and the module provides the encryption keys to clients. The encryption key is used to encrypt the information when generating a multi-signature wallet, for high security. To verify whether the encryption key is correct or not, we use zero-knowledge proof [23], which is a method that allow data be verified without revealing that data. If the proofs are not valid, the key generation and signing process is stopped. This preprocessing guarantees that a multi-signature is generated and a transaction is signed securely.

3.3 Generating a multi-signature wallet

To transfer digital assets on a blockchain, the clients create a multi-signature wallet. In our scheme, the multi-signature wallet operates as a single-signature wallet to reduce the validation time. To do this, we generate a common public key and shared keys as follows. As mentioned in Section 3.2, clients perform a preprocessing step before generating a multi-signature wallet. After the step, all the clients generate private/public keys via the *key generation module* and share and bind the public key among themselves by using a commitment scheme via the *communication module*. A commitment scheme is a cryptographic primitive that allows one to commit to a chosen value (i.e., public key) while keeping it hidden from others, with the ability to reveal the committed value later [6]. If all the

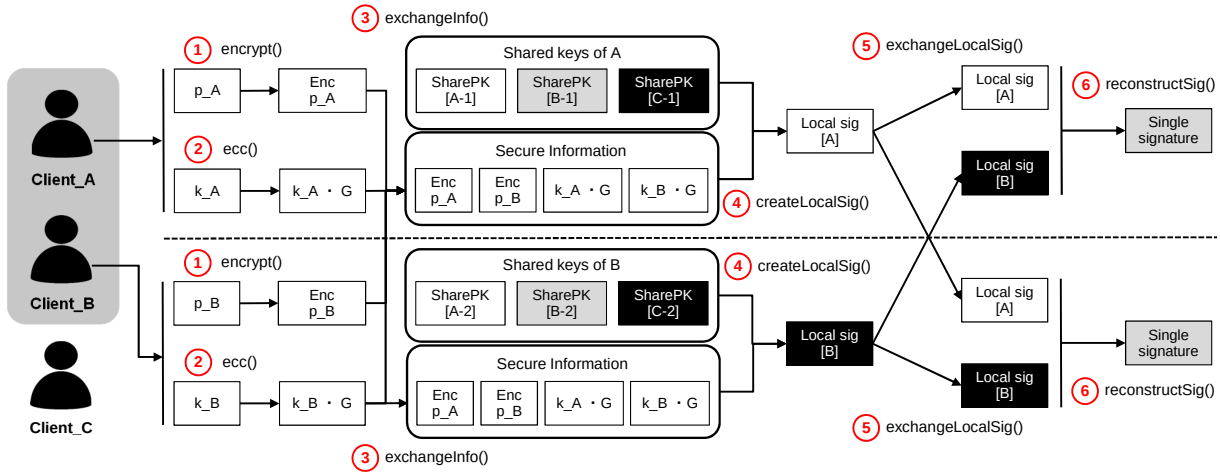


Figure 5: A signing process for 2 of 3 multi-signature wallet (p: random number, k: random number, G: generate point, sig: signature)

public keys are valid, each client generates a common public key with the public keys. The common public key represents the multi-signature wallet and is generated by the *key generation module*.

To generate shared keys, each client divides its own private key to N shared keys with the threshold (T) and the number of participants (N). For example, Figure 3 shows a 2 of 3 multi-signature wallet generating shared keys of a multi-signature wallet. Each client (i.e., client A, client B, and client C) first splits its own private key as three shared keys (SharedPK) via `dividePK()` in the *key generation module* (①). The shared keys are used to sign a transaction. Second, each client encrypts the shared keys with participants' encryption keys via `encrypt()` in the *encryption module* (②). For example, client A encrypts the shared key (sharedPK [A-2]) to be transferred to client B with client B's encryption key. Similarly, the shared key (sharedPK [A-3]) to be transferred to client C is encrypted client C's encryption key. The encrypted shared key [A-2] (enc sharedPK [A-2]) can be opened only by client B, and the encrypted shared key [A-3] (enc sharedPK [A-3]) can be opened only by client C.

After the encryption, each client distributes encrypted shared keys to their associated clients via `share()` in the *communication module* (③). Next, the clients decrypt the transferred shared keys with their decryption keys via `decrypt()` in the *decryption module* (④). Note that the clients exchange each shared key by using zero-knowledge proof (ZKP). Therefore, all clients verify whether the shared keys are valid without actually transferring the shared keys via *validation module*. If the proofs are not valid, the key generation process is stopped.

3.4 Signing transaction via multi-signature and Bloom-filter

As mentioned in Section 3.2, clients perform a preprocessing step before signing a transaction using multi-signature. After the preprocessing step, to identify the participant of a transaction, clients generate a Bloom-filter with the participants' public keys via the *transaction generation module*. As mentioned Section in 2.4, because Bloom-filter has a false-positive feature, all clients should decide a Bloom-filter size with an array of m-bits and k hash functions. We calculate the m-bits size and the number of k hash functions by using an equation 2, the fixed number of participants, and a false positive rate. We set the probabilistically safe state as 2^{-20} and the maximum number of participants in a transaction as ten. As a result, the bloom-filter has from 29 bit up to 289 bits of size and 20 hash functions, which is the optimal number of the bloom-filter.

Figure 4 shows generating a bloom-filter of a transaction when two clients (client A and B) participate in a signing process of a 2 of 3 multi-signature wallet. First, each client calculates m-bits size with the number of participants (i.e., 2). In this case, the Bloom-filter size is 58 bits. Next, each client calculates its own 20 hash values (`H1`∼`H20`) with each participant's public key (i.e., A_pub and B_pub) and sets the corresponding bit to 1 for each hash value as the bit array index (①). The generated Bloom-filter is exchanged and validated by each client using a commitment scheme. For example, to identify whether client A signs in the transaction, all the clients calculate their own 20 hash values (`H1`∼`H20`) with client A's public key. They then check the corresponding bit for each hash value as the bit array index (②). If all values are 1, it indicates that client A has participated in the signing of the transaction. This process is performed for client B in the same manner. After verifying that the Bloom-filter indicates that all clients (clients A and B) have participated in the signing process, the clients store the Bloom-filter in the transaction. If the validation fail, the signing process is stopped.

After generating the Bloom-filter, the clients perform the process of signing the transaction. In our scheme, we distribute the authority of signing a transaction by using the T-ECDSA for high security. For this, we use multi-party computation (MPC) in the T-ECDSA which is a method for creating a single signature by distributing the authority into multiple clients. In this process, to distribute the authority and hide information, we generate random numbers and a local signature for each client, respectively. Figure 5 shows the signing process of a 2 of 3 multi-signature wallet. Clients A and B first generate their own random numbers (k and p), which are used to generate a single signature via the T-ECDSA. For example, clients A and B generate random numbers such as p_A/k_A and

p_B/*k_B*, respectively. The parameter *k* is used to generate a local signature and *p* is used to hide its own shared keys in the signing process. Next, the clients encrypt their own *p* via `encrypt()` (①). Each client then calculates a point of elliptic curve with its own random *k* and generation point *G* via `ecc()` (②).

To generate a local signature, the clients exchange the encrypted *p* (i.e., *enc_p_A* and *enc_p_B*) and the points (i.e., *k_A·G* and *k_B·G*) with each other via `exchangeInfo()` (③). The clients load the generated information (shared keys, private key, etc) mentioned in Section 3.3. Subsequently, the clients generate their local signatures (*local_sig_A* and *local_sig_B*) with the shared keys and secure information (i.e., *enc_p_A*, *enc_p_B*, *k_A·G*, and *k_B·G*) via `createLocalSig()` (④). Each local signature is part of a single signature and contains information of each participant's shared keys encrypted. The signatures are used to reconstruct a complete single signature. After creating the local signature, the clients exchange their local signature via `exchangeLocalSig()` (⑤). Furthermore, each client completes to sign the transaction by reconstructing the local signatures to obtain a single signature via `reconstructSig()` (⑥). A signed transaction is generated for each participant, and the transaction is sent to the blockchain by client A who creates a channel in the preprocessing step of Section 3.2 by the wallet rule.

Unlike Shamir's secret sharing [24] scheme, the process of generating a single signature is more secure because in the process, each local signature, rather than a common private key, is reconstructed for signing the transaction. Therefore, each participant cannot acquire secure information such as private keys and shared keys, and a malicious server cannot sign another transaction with the information. In contrast to other multi-signature wallets [3, 4], the proposed wallet does not include any participant's public key or wallet address since it uses a Bloom-filter, thereby providing more privacy.

3.5 Identifying a participant of a transaction

After signing a transaction, the proposed multi-signature wallet identifies who sign the transaction in order to hold clients accountable for the transfer of digital assets. For this, the *validation module* in a client downloads the transaction data from blockchains for identifying a participant of a transaction. The module extracts the Bloom-filter from the transaction and checks who signs the transaction via the Bloom-filter. For example, as shown in Figure 4, the Bloom-filter identifies whether client A signs a transaction. If all the values of index corresponding to the hash value of client A's public key are 1, we can identify that client A sign the transaction. Otherwise, we can identify that client A does not sign the transaction. Through these processes, we can identify the participant via the Bloom-filter values, without exposing private information.

4 EVALUATION

4.1 Experimental setup

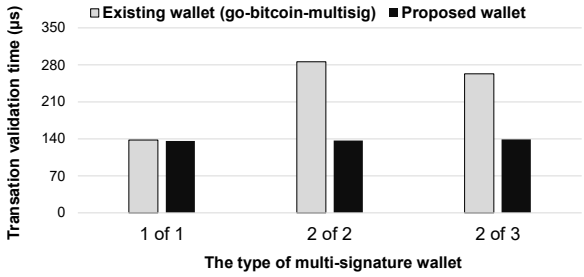
In our evaluation, we use a server node comprising two Intel Xeon E5-2683 processors (total 32 cores), 64 GiB DRAM, and running Ubuntu 16.04.5 LTS. We use ten client nodes, and each client includes an Intel i7 processor and 32 GiB DRAM running macOS. We evaluate the existing multi-signature wallets go-bitcoin-multisig [2] and MultiSigWallet [4] and our proposed wallet on two blockchains (i.e., Bitcoin and Ethereum). We compare the proposed wallet with go-bitcoin-multisig supporting multi-signature and using pay to script hash (P2SH) for the Bitcoin protocol. We also compare our multi-signature wallet with MultiSigWallet, which uses a smart contract, since the Ethereum blockchain does not support multi-signature as a protocol. We evaluate the transaction validation time, transaction size, and transaction fees (cost), and examine the privacy. Note that we measure the performance for 1 of 1, 2 of 2, and 2 of 3 since Bitcoin supports a multi-signature up to 2 of 3. In Ethereum, generally, the number of participants is ten, and therefore, we measure the performance in the case of 1 of 1, 2 of 2, 2 of 3, 3 of 4, 4 of 5, and 5 of 10.

4.2 Performance results

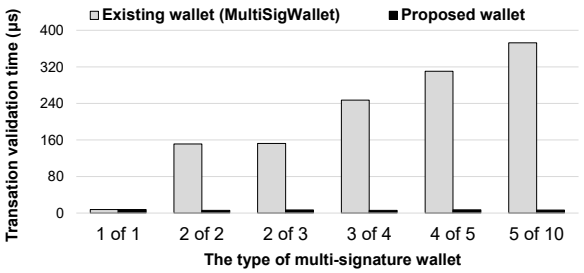
Transaction validation time. Figure 6(a) shows the transaction validation time of one of the existing wallet (go-bitcoin-multisig) and our proposed wallet on the Bitcoin blockchain. We compare multi-signature until 2 of 3 participants since Bitcoin supports multi-signature up to 2 of 3 by default. In the case of 1 of 1, the performance as a baseline of the existing and proposed schemes is equal since both wallets use the general single signature scheme. For the multi-signature types 2 of 2 and 2 of 3, the proposed wallet shows performance improvement by up to 2x and 1.89x compared with the existing wallet, respectively. The transaction signed by the existing wallet require more validation time since multiple signatures have to be validated. By contrast, since the proposed wallet signs a transaction with a single signature, the verification time is the same as that of a single signature scheme. Therefore, the proposed wallet shows higher validation performance than the existing multi-signature wallet.

Figure 6(b) shows the transaction validation time for the other existing wallet (MultiSigWallet) and our proposed wallet on Ethereum blockchain. Likewise, the performance of the wallet is equal for the existing and proposed schemes for 1 of 1. For the multi-signature types 2 of 2, 2 of 3, 3 of 4, 4 of 5, and 5 of 10, the proposed wallet shows performance improvements by up to 24.7x, 22.9x, 41.1x, 44.3x, and 58.3x compared with the existing wallet, respectively. MultiSigWallet generates multiple smart contract transactions such as submit and confirm transactions, and these transactions should be validated by executing the smart contract on Ethereum, which increases the validation time. Since it generates only a single transaction, the proposed wallet shows higher validation performance than the existing wallet.

Transaction size. Figure 7(a) shows the transaction size of the existing wallet (go-bitcoin-multisig) and our proposed wallet on the Bitcoin blockchain. The proposed wallet reduces the transaction size by up to 1.58x and 1.86x compared with the existing wallet for the multi-signature types 2 of 2 and 2 of 3, respectively. This is because the existing wallet signs a transaction with multiple signatures. Moreover, a transaction of the existing wallet involves all participants' public keys for validating a transaction. Therefore, as the number of participants increases, the transaction size of the existing wallet increases. By contrast, the proposed wallet signs a transaction with a single signature, a Bloom-filter, and a common public key for validating a transaction, which reduces the transaction size.

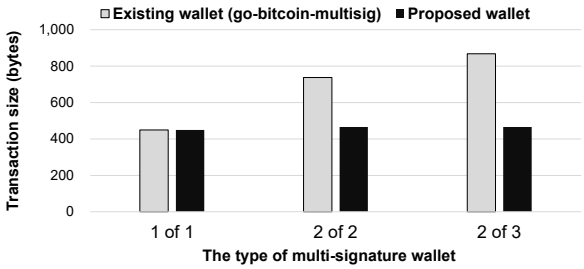


(a) Bitcoin

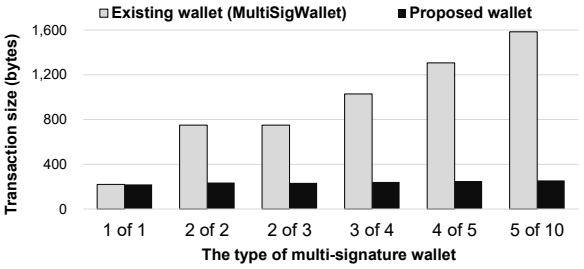


(b) Ethereum

Figure 6: Transaction validation time



(a) Bitcoin



(b) Ethereum

Figure 7: Transaction size

Figure 7(b) shows the transaction size of the existing wallet (MultiSigWallet) and our proposed wallet on Ethereum blockchain. The proposed wallet reduces the transaction size by up to 3.15x, 3.2x, 4.24x, 5.2x, and 6.18x compared with the existing wallet for the multi-signature types 2 of 2, 2 of 3, 3 of 4, 4 of 5, and 5 of 10, respectively. As the number of participants increases, the total transaction size of the existing wallet increases since the number of multiple transactions increases. By contrast, in the case of the proposed wallet, the transaction size is almost constant or increases only slightly. The reason is that the wallet generates a single transaction even though the number of participants increases and the size of bloom-filter increases slightly according to the number of participants.

Transaction fee. A client should pay a transaction fee when transmitting a digital asset on a blockchain. In the case of Bitcoin, the transaction fee depends on the transaction confirmation time[4] Before the final transaction fee is determined, an optimal transaction fee per byte is decided by considering the Bitcoin network status and transaction confirmation time by using a Bitcoin fee calculator [5]. The final transaction fee is the decided from the optimal transaction fee and the transaction size. Figure 8(a) shows the transaction fee of the existing wallet (go-bitcoin-multisig) and the proposed wallet on the Bitcoin blockchain. The proposed wallet reduces a transaction fee by up to 1.58x and 1.86x compared with the existing wallet for the multi-signature types 2 of 2 and 2 of 3, respectively. Thus, we could reduce the transaction fee by reducing the transaction size through the use of a single signature.

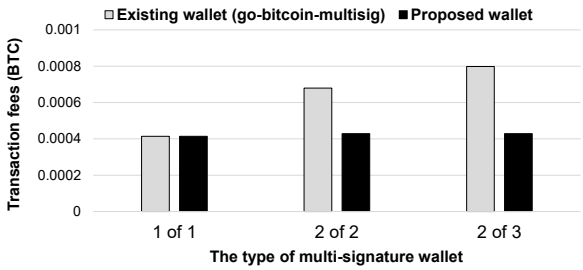
[4]The transaction confirmation time is the time interval from the beginning and the approval of the transaction on a blockchain. We set this confirmation time as 60 minutes on the basis of a previous study [22].

In the case of Ethereum, the transaction fee depends on gas usage and gas price[5]. Before determining the transaction fee, an optimal gas price is decided by considering Ethereum network status and the transaction confirmation time via an Ethereum gas station [8]. The transaction fee is then determined by the optimal gas price and gas usage. The gas usage is charged cumulatively whenever a smart contract is executed or extra data is included. Figure 8(b) shows the transaction fees of the existing wallet (MultiSigWallet) and our proposed wallet on the Ethereum blockchain. The proposed wallet reduces the transaction fee by up to 10.7x, 11.6x, 15.2x, 20.7x, and 27x compared with the existing wallet for the multi-signature types 2 of 2, 2 of 3, 3 of 4, 4 of 5, and 5 of 10, respectively. The existing wallet creates multiple transactions and executes a smart contract to sign a transaction, which incurs more gas usage. By contrast, our scheme does not require the execution of a smart contract. Although additional data is obtained from Bloom-filter, it hardly affects the gas usage. Consequently, the proposed wallet incurs a lower transaction fee than the existing wallet.

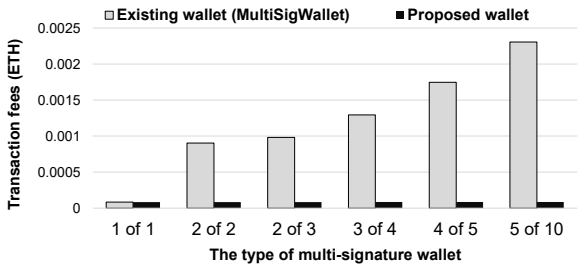
4.3 Discussion of privacy

Any client can access data on public blockchains such as Bitcoin and Ethereum. Thus, there can be privacy problems for the existing wallets on public blockchains since the blockchains can store participants' information such as public key and wallet address. For example, in the case of MultiSigWallet, a signed transaction on the blockchain includes the associated participants' wallet addresses. In the case of go-bitcoin-multisig, a signed transaction on the blockchain includes all participants' public keys that are required to verify the transaction. Consequently, in both existing schemes, there can be privacy problems since the participants' information (public keys or wallet addresses) on the blockchain can

[5]The unit of transaction fee in Ethereum is gas



(a) Bitcoin



(b) Ethereum

Figure 8: Transaction fees (BTC: bitcoin, ETH: ethereum)

be exposed to any client. In the proposed wallet, the signed transaction includes a common public key and a Bloom-filter instead of participants' information. Therefore, the proposed wallet can provide more privacy than the existing wallets by preventing to the exposure of participants information.

5 RELATED WORK

Previous studies [17–19] have investigated the threshold signature scheme for improving the signature performance. Gennaro et al. [17] present the first protocol that supports a multi-party threshold ECDSA with an efficient distributed key generation. Thus, it reduces the execution time and data transfer time for key generation. Goldfeder et al. [19] introduce the first practical t-of-n threshold signature scheme compatible with Bitcoin's ECDSA signatures. They show how Bitcoin wallets use a threshold signature scheme for signing and verifying a transaction. Gennaro et al. [18] introduce a threshold-optimal digital signature algorithm (DSA) scheme for minimizing the number of servers targeted by a malicious attacker. Our study is in line with existing approaches [17–19] in terms of exploiting the threshold signature in the blockchain wallet system to decrease the validation and signature times. However, we furthermore reduce the transaction size and improve privacy by using a Bloom-filter apart form improving the validation and signature performance.

There have been studies on investigating the hardware wallets. Hardware wallets (e.g., Ledger [10], Trezor [11], and Keepkey [9]) allow to store the private key in an offline and secure storage. They can operate directly on a personal computer or mobile phone through Bluetooth or by being plugged in via USB instead of the internet. Thus, the private keys would remain completely secure even if the hardware wallet connects to a computer infected with a virus. In addition, for higher security, a multi-signature provides multiple keys to authorize a Bitcoin transaction, rather than a single signature from one key. Our study is inline with these approaches [9–11] in terms of improving the security of wallets. Furthermore, we improve higher security by distributing authorities of the wallets against a single point attack.

There have been studies on improving the security of blockchain wallets. MultiSigWallet [4] provides an m-out-of-n multi-signature wallet with smart contracts on Ethereum. It executes basically smart contracts able to store tokens and requiring that the issued transaction needs to accept some requirements, including a specific signing configuration. BitGo [3] presents a 2-of-3 multi-signature wallet and a new type of bitcoin address introduced as part of BIP 16 (Bitcoin Improvement Proposal 16). It provides more secure Bitcoin addresses compared with traditional Bitcoin addresses by using the multi-signature wallet. Our study is inline with these approaches [3, 4, 9–11] in terms of improving security of wallets. In particular, we focus on providing a higher performance for multi-signature on blockchains by reducing the number of transactions to be validated and the transaction size.

6 CONCLUSION

We investigate blockchain wallets that support a digital signature in blockchains, and found that while a multi-signature wallet provides higher security than a single-signature wallet, it increases the tramsactopm validation time and transaction size. Furthermore, the use of a multi-signature wallet, requires the modification of the blockchain protocol. To overcome these issues, we develop a new multi-signature wallet with high performance and a small transaction size by using a T-ECDSA and a Bloom-filter. We implement the proposed multi-signature wallet using a Multi-party ECDSA and evaluate the wallet's performance for Bitcoin and Ethereum. Our experimental results show that the proposed multi-signature wallet reduces the validation time and transaction size than other existing multi-signature wallets.

7 ACKNOWLEDGMENTS

This work was partially supported by the Technology development Program of MSS (S2878336), the National Research Foundation of the Korea government (MSIT) (NRF-2018R1C1B5085640), the National Research Foundation (NRF) grant (NRF-2016M3C4A7952587), BK21 FOUR Intelligence Computing(Dept. of Computer Science and Engineering, SNU) funded by National Research Foundation of Korea (NRF) (4199990214639), and Seoul R&D Program (CY20038). (Corresponding Author: Yongseok Son)

REFERENCES

[1] 2012. *BIP: 16.* https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki.
[2] 2014. *bitcoin-multisig.* https://github.com/soroushjp/go-bitcoin-multisig.
[3] 2014. *BitGo.* https://www.bitgo.com/.
[4] 2019. *MultiSigWallet.* https://github.com/Gnosis/MultiSigWallet.
[5] 2020. *Bitcoin Fee Calculator Estimator.* https://www.buybitcoinworldwide.com/fee-calculator.
[6] 2020. *Commitment scheme.* https://en.wikipedia.org/wiki/Commitment_scheme.
[7] 2020. *Elliptic-curve cryptography.* https://en.wikipedia.org/wiki/Elliptic-curve_cryptography.
[8] 2020. *Eth gas station.* https://ethgasstation.info/calculatorTxV.php.
[9] 2020. *Keepkey.* https://shapeshift.com/keepkey.
[10] 2020. *Ledger.* https://www.ledger.com/.
[11] 2020. *Trezor.* https://trezor.io/.
[12] 2020. *What is a blockchain address?* https://wirexapp.com/help/article/what-is-a-blockchain-address-0068.

[13] 2020. *Wikipedia: Multisignature*. https://en.wikipedia.org/wiki/Multisignature.
[14] 202020. *multi-party-ecdsa*. https://github.com/KZen-networks/multi-party-ecdsa.
[15] Burton H Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (1970), 422–426.
[16] Vitalik Buterin et al. 2014. A next-generation smart contract and decentralized application platform. *white paper* 3, 37 (2014).
[17] Rosario Gennaro and Steven Goldfeder. 2018. Fast multiparty threshold ECDSA with fast trustless setup. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1179–1194.
[18] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. 2016. Threshold-optimal DSA/ECDSA signatures and an application to Bitcoin wallet security. In *International Conference on Applied Cryptography and Network Security*. Springer, 156–174.
[19] Steven Goldfeder, Rosario Gennaro, Harry Kalodner, Joseph Bonneau, Joshua A Kroll, Edward W Felten, and Arvind Narayanan. 2015. Securing Bitcoin wallets via a new DSA/ECDSA threshold signature scheme. In *et al.*
[20] Ali Al Imem. 2015. Comparison and evaluation of digital signature schemes employed in NDN network. *arXiv preprint arXiv:1508.00184* (2015).
[21] Don Johnson, Alfred Menezes, and Scott Vanstone. 2001. The elliptic curve digital signature algorithm (ECDSA). *International journal of information security* 1, 1 (2001), 36–63.
[22] Satoshi Nakamoto. 2019. *Bitcoin: A peer-to-peer electronic cash system*. Technical Report. Manubot.
[23] Charles Rackoff and Daniel R Simon. 1991. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In *Annual International Cryptology Conference*. Springer, 433–444.
[24] Adi Shamir. 1979. How to share a secret. *Commun. ACM* 22, 11 (1979), 612–613.
[25] Victor Shoup. 2000. Practical threshold signatures. In *International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 207–220.
[26] Jesse Yli-Huumo, Deokyoon Ko, Sujin Choi, Sooyong Park, and Kari Smolander. 2016. Where is current research on blockchain technology?—a systematic review. *PloS one* 11, 10 (2016), e0163477.